

Listas

September 1, 2026

1 Listas em Python

Este notebook apresenta listas para quem está começando em Python.

Ao final, você deve conseguir:

- criar listas vazias, com dados e listas dentro de listas;
- acessar, alterar e percorrer elementos;
- consultar informações com `len`, `sum`, `min`, `max`, `in`, `index` e `count`;
- adicionar, ordenar e remover itens;
- resolver um problema simples usando listas paralelas.

1.1 Roteiro

1. O que é uma lista
2. Criando listas
3. Acessando e alterando itens
4. Listas dentro de listas
5. Percorrendo listas
6. Consultando informações
7. Adicionando e ordenando itens
8. Removendo itens
9. Exercícios

1.2 1. O que é uma lista?

Uma lista guarda vários valores em uma única variável. Em Python, usamos colchetes `[]`.

Cada item tem uma posição, chamada índice. O primeiro índice é 0.

```
[1]: alunos = ["Ana", "Bruno", "Carlos"]

print(alunos)
print("Primeiro aluno:", alunos[0])
print("Segundo aluno:", alunos[1])
print("Terceiro aluno:", alunos[2])
```

```
['Ana', 'Bruno', 'Carlos']
Primeiro aluno: Ana
Segundo aluno: Bruno
Terceiro aluno: Carlos
```

1.3 2. Criando listas

Podemos criar uma lista vazia quando ainda não temos dados, ou criar uma lista já preenchida.

```
[2]: lista_vazia = []
      notas = [3.5, 7.8, 1.5, 9, 10, 2.6]
      nomes = ["João", "Pedro", "Maria"]
      lista_mista = [1, "Diego", 7.8, True]

      print("Lista vazia:", lista_vazia)
      print("Notas:", notas)
      print("Nomes:", nomes)
      print("Lista mista:", lista_mista)
```

```
Lista vazia: []
Notas: [3.5, 7.8, 1.5, 9, 10, 2.6]
Nomes: ['João', 'Pedro', 'Maria']
Lista mista: [1, 'Diego', 7.8, True]
```

1.4 3. Acessando itens e fatias

Use o índice para acessar um item. Use uma fatia para pegar parte da lista.

Lembrete: em `lista[inicio:fim]`, o índice `fim` não entra no resultado.

```
[3]: notas = [3.5, 7.8, 1.5, 9, 10, 2.6]

      print("Primeira nota:", notas[0])
      print("Última nota:", notas[-1])
      print("Três primeiras notas:", notas[0:3])
      print("Da terceira até o final:", notas[2:])
```

```
Primeira nota: 3.5
Última nota: 2.6
Três primeiras notas: [3.5, 7.8, 1.5]
Da terceira até o final: [1.5, 9, 10, 2.6]
```

1.5 4. Alterando itens

Listas são mutáveis. Isso significa que podemos trocar o valor de uma posição.

```
[4]: alunos = ["Ana", "Bruno", "Carlos"]

      alunos[0] = "Felipe"

      print(alunos)
```

```
['Felipe', 'Bruno', 'Carlos']
```

1.6 5. Listas dentro de listas

Uma lista pode guardar outra lista. Isso é útil para representar dados agrupados, como uma tabela simples.

```
[5]: turma = [  
    ["Ana", 8.5, 9.0],  
    ["Bruno", 6.0, 7.5],  
    ["Carlos", 9.0, 8.0],  
]  
  
print("Primeiro registro:", turma[0])  
print("Nome do primeiro aluno:", turma[0][0])  
print("Segunda nota do terceiro aluno:", turma[2][2])
```

```
Primeiro registro: ['Ana', 8.5, 9.0]  
Nome do primeiro aluno: Ana  
Segunda nota do terceiro aluno: 8.0
```

1.7 6. Percorrendo listas diretamente

Quando queremos usar cada elemento da lista, podemos fazer um `for` diretamente sobre a lista.

```
[6]: alunos = ["Ana", "Bruno", "Carlos"]  
  
for aluno in alunos:  
    print("Olá,", aluno)
```

```
Olá, Ana  
Olá, Bruno  
Olá, Carlos
```

1.8 7. Percorrendo listas com `range` e índice

Quando precisamos da posição do item, podemos percorrer os índices com `range(len(lista))`.

```
[7]: notas = [8.5, 6.0, 9.0]  
  
for indice in range(len(notas)):  
    print("Índice:", indice, "| Nota:", notas[indice])
```

```
Índice: 0 | Nota: 8.5  
Índice: 1 | Nota: 6.0  
Índice: 2 | Nota: 9.0
```

1.9 8. Percorrendo duas listas relacionadas

Às vezes uma lista guarda os nomes e outra guarda as notas. O índice liga uma informação à outra.

```
[8]: alunos = ["Ana", "Bruno", "Carlos"]
      notas = [8.5, 6.0, 9.0]

      for indice in range(len(alunos)):
          print(alunos[indice], "tirou", notas[indice])
```

Ana tirou 8.5
Bruno tirou 6.0
Carlos tirou 9.0

1.10 9. Consultando informações da lista

Algumas funções e métodos ajudam a responder perguntas sobre a lista.

```
[9]: notas = [8.5, 6.0, 9.0, 6.0]

      print("Quantidade de notas:", len(notas))
      print("Soma das notas:", sum(notas))
      print("Menor nota:", min(notas))
      print("Maior nota:", max(notas))
      print("Média:", sum(notas) / len(notas))
```

Quantidade de notas: 4
Soma das notas: 29.5
Menor nota: 6.0
Maior nota: 9.0
Média: 7.375

1.11 10. Verificando se um item existe

O operador `in` responde se um valor aparece na lista.

```
[10]: opcoes = ["adição", "subtração", "multiplicação", "divisão"]
      opcao = "adição"

      if opcao in opcoes:
          print("Você escolheu uma opção válida!")
      else:
          print("Você escolheu uma opção inválida!")
```

Você escolheu uma opção válida!

1.12 11. Descobrindo posição e quantidade

Use `index` para descobrir a primeira posição de um valor.

Use `count` para contar quantas vezes um valor aparece.

```
[11]: notas = [8.5, 6.0, 9.0, 6.0]
```

```
print("Primeira posição da nota 6.0:", notas.index(6.0))
print("Quantas vezes 6.0 aparece:", notas.count(6.0))
```

Primeira posição da nota 6.0: 1

Quantas vezes 6.0 aparece: 2

Antes de usar `index`, é mais seguro verificar se o valor existe na lista.

```
[12]: alunos = ["Ana", "Bruno", "Carlos"]
      procurado = "Daniel"

      if procurado in alunos:
          print("Posição:", alunos.index(procurado))
      else:
          print(procurado, "não está na lista.")
```

Daniel não está na lista.

1.13 12. Adicionando itens

Use `append` para adicionar no final.

Use `insert` para adicionar em uma posição específica.

```
[13]: alunos = ["Ana", "Bruno", "Carlos"]

      alunos.append("Daniel")
      alunos.insert(1, "Bianca")

      print(alunos)
```

['Ana', 'Bianca', 'Bruno', 'Carlos', 'Daniel']

1.14 13. Ordenando listas

O método `sort` altera a própria lista.

```
[14]: notas = [3.5, 7.8, 1.5, 9, 10, 2.6]

      notas.sort()
      print("Crescente:", notas)

      notas.sort(reverse=True)
      print("Decrescente:", notas)
```

Crescente: [1.5, 2.6, 3.5, 7.8, 9, 10]

Decrescente: [10, 9, 7.8, 3.5, 2.6, 1.5]

1.15 14. Removendo itens

Existem várias formas de remover itens. A melhor escolha depende do que você sabe:

- Se você souber qual é o índice, use `del`;
- Se você souber qual é o valor? Use `remove`;
- Se quiser apagar tudo? Use `clear`.

```
[22]: alunos = ["Ana", "Bruno", "Carlos", "Daniel"]

del alunos[0]
print("Depois de del alunos[0]:", alunos)

alunos.remove("Daniel")
print("Depois do remove:", alunos)
```

Depois de del alunos[0]: ['Bruno', 'Carlos', 'Daniel']

Depois do remove: ['Bruno', 'Carlos']

Também é possível remover uma fatia, ou seja, um intervalo de posições.

```
[16]: numeros = [10, 20, 30, 40, 50, 60]

del numeros[1:4]

print(numeros)
```

[10, 50, 60]

1.16 15. Erros comuns

Tentar acessar uma posição que não existe gera erro.

Antes de acessar, confira o tamanho da lista com `len`.

```
[17]: alunos = ["Ana", "Bruno"]
indice = 2

if indice < len(alunos):
    print(alunos[indice])
else:
    print("Esse índice não existe na lista.")
```

Esse índice não existe na lista.

1.17 Exercício 1

Crie uma lista com quatro sabores de sorvete e outra lista com a quantidade vendida de cada sabor.

Depois, mostre cada sabor junto com sua quantidade.

```
[18]: # Tente resolver aqui.
sabores = ["morango", "chocolate", "baunilha", "limão"]
vendas = [12, 18, 7, 10]
```

```
for indice in range(len(sabores)):
    print(sabores[indice], "-", vendas[indice], "vendas")
```

```
morango - 12 vendas
chocolate - 18 vendas
baunilha - 7 vendas
limão - 10 vendas
```

1.18 Exercício 2: controle de vendas de sorvetes

Uma sorveteria quer registrar vendas por sabor.

Regras:

- sabores válidos: morango, chocolate, baunilha e limão;
- para cada venda válida, some a quantidade ao sabor correto;
- se o sabor não existir, mostre que ele é inválido;
- ao final, mostre total vendido, sabor mais vendido e sabor menos vendido.

Aqui vamos usar uma lista de vendas pronta para o notebook rodar sem depender de `input`.

```
[19]: sabores = ["morango", "chocolate", "baunilha", "limão"]
vendas = [0, 0, 0, 0]

registros = [
    ["morango", 3],
    ["chocolate", 5],
    ["baunilha", 2],
    ["chocolate", 4],
    ["pistache", 1],
    ["limão", 6],
]

for registro in registros:
    sabor = registro[0]
    quantidade = registro[1]

    if sabor in sabores:
        indice_sabor = sabores.index(sabor)
        vendas[indice_sabor] = vendas[indice_sabor] + quantidade
    else:
        print("Sabor inválido:", sabor)

print("Sabores:", sabores)
print("Vendas:", vendas)
```

Sabor inválido: pistache

Sabores: ['morango', 'chocolate', 'baunilha', 'limão']

Vendas: [3, 9, 2, 6]

Agora calculamos os resultados finais usando as listas.

```
[20]: total_vendas = sum(vendas)
sabor_mais_vendido = sabores[vendas.index(max(vendas))]
sabor_menos_vendido = sabores[vendas.index(min(vendas))]

print("Total de vendas:", total_vendas)
print("Sabor mais vendido:", sabor_mais_vendido)
print("Sabor menos vendido:", sabor_menos_vendido)
```

Total de vendas: 20
Sabor mais vendido: chocolate
Sabor menos vendido: baunilha

1.19 Versão opcional com input

Use esta versão quando quiser digitar os dados manualmente.

Ela está comentada para não travar a execução automática do notebook.

```
[21]: # sabores = ["morango", "chocolate", "baunilha", "limão"]
# vendas = [0, 0, 0, 0]
#
# sabor = input("Qual o sabor? ")
#
# while sabor != "sair":
#     quantidade = int(input("Qual a quantidade? "))
#
#     if sabor in sabores:
#         indice_sabor = sabores.index(sabor)
#         vendas[indice_sabor] = vendas[indice_sabor] + quantidade
#     else:
#         print("Sabor inválido!")
#
#     sabor = input("Qual o sabor? ")
#
# print("Total de vendas:", sum(vendas))
```

1.20 Desafio extra

Altere o exercício da sorveteria para mostrar também a média de vendas por sabor.